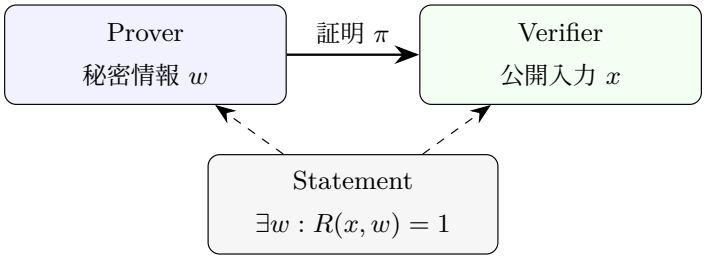
ゼロ知識証明に関するリサーチ論文

基礎・セキュリティ評価・方式比較・ユースケース

公的研究機関による民間組織向け技術調査資料案



作成日：2026 年 4 月 27 日

本資料は技術調査・研究用途のドラフトであり、特定製品または特定実装の採用を推奨するものではない。

要旨

ゼロ知識証明（Zero-Knowledge Proof; ZKP）は、ある主張が正しいことを、その主張を成り立たせる秘密情報を開示せずに検証可能にする暗号技術である。本稿は、ZKP の基礎概念、歴史、セキュリティ定義、主要方式の比較、実用例を体系的に整理する。特に、Groth16、BBS 署名、Bulletproofs、One-out-of-Many proof、PLONK、ZK-STARK、Ligero、Aurora、Virgo を、証明対象、安全性仮定、信頼セットアップ、ポスト量子性、性能、実装成熟度、ユースケース適合性の観点から比較する。

本稿の中心的な整理は、ZKP を単一技術として扱わず、用途と信頼仮定の異なる方式群として捉える点にある。短い証明と高速検証を重視する場合にはペアリングベースの SNARK が有力であり、属性証明には BBS 署名、範囲証明には Bulletproofs、透明セットアップと長期安全性を重視する場合には STARK 系や Ligero、Aurora、Virgo が候補となる。したがって、導入判断では、証明したい statement、秘匿したい witness、検証者の立場、公開検証の要否、信頼セットアップの許容性、証明保存期間、量子耐性要求を明確にする必要がある。

目次

要旨 i

第 1 章 はじめに 1

1.1 ゼロ知識証明とは何か 1

1.2 ゼロ知識証明の歴史 2

第 2 章 ゼロ知識証明の基本 5

2.1 ゼロ知識証明の意義 5

2.2 Statement と Witness 6

2.3 ゼロ知識証明の 3 つの特性 7

2.4 インタラクティブ証明と非インタラクティブ証明 8

2.5 現代的ゼロ知識証明の概観 8

第 3 章 ゼロ知識証明のセキュリティ特性と評価軸 11

3.1 ゼロ知識証明におけるセキュリティ定義 11

3.2 スキーム比較時の評価軸 14

第 4 章 各 ZKP 方式の比較 18

4.1 比較の前提：安全性仮定、用途、方式階層 18

4.2 代数的仮定ベースの ZKP・ZKPoK 19

4.3 透明セットアップ・ハッシュ／ PCP・IOP ベースの ZKP 22

4.4 各方式の横断比較 24

第 5 章 ゼロ知識証明のユースケースと実用例 26

5.1 デジタル ID・資格証明・選択的開示 26

5.2 認証・アクセス制御 27

5.3 金融・会計・監査 28
5.4 ブロックチェーン・分散台帳 28
5.5 検証可能計算・クラウド委託計算 29
5.6 サプライチェーン・取引条件・契約遵守 30
5.7 電子投票・公開監査 30
5.8 AI・機械学習：zkML 31
5.9 ユースケース別の方式選定 31
5.10 実装パターンと設計上の論点 32

第 1 章　はじめに

デジタル化された業務では、本人確認、取引承認、契約履行、監査、サプライチェーン管理、クラウド計算、AI 利用など、多くの場面で「ある主張が正しいこと」を第三者に示す必要がある。従来、この確認は、個人情報、営業秘密、取引条件、内部データ、アルゴリズム、鍵情報などの開示を伴うことが多かった。しかし、データ保護規制、サイバーセキュリティ、競争上の機密保持、越境データ移転、委託先管理の重要性が高まるにつれ、検証可能性と情報最小化を同時に満たす技術への需要が高まっている。

ゼロ知識証明は、この課題に対する暗号学的な基盤技術である。NIST は ZKP を、証明者が検証者に対して、ある statement が真であることを、その真偽を示す情報以上を与えずに納得させる暗号方式として定義している [1]。また NIST の Privacy-Enhancing Cryptography プロジェクトは、ZKP を、数学的 statement の真実性を、その真実性を見いだすために有用であった追加情報を明らかにせずに証明する主要技術として位置づけている [2]。

ZKP の意義は、秘密を隠すことだけではない。より本質的には、従来であれば信頼、契約、監査、または中央管理に依存していた確認行為を、暗号学的な検証可能性に置き換える点にある。利用者が年齢要件を満たしていることを生年月日そのものを開示せずに示す、企業が一定の準拠条件を満たすことを内部データを開示せずに証明する、クラウド事業者が計算結果の正しさを入力や計算過程を公開せずに証明する、といった応用が考えられる。

ただし、ZKP は万能の秘匿技術ではない。何を証明するかを表す statement 設計、秘密情報を表す witness の管理、計算を証明可能な形式へ変換する回路化、証明生成コスト、検証コスト、信頼セットアップの有無、耐量子性、実装上のサイドチャネル、既存システムとの統合可能性を総合的に評価する必要がある。本稿では、ZKP を事業システムまたは社会基盤システムへ導入する際の判断材料を提供するため、基礎概念、セキュリティ定義、主要方式、実用例を順に整理する。

1.1　ゼロ知識証明とは何か

ゼロ知識証明とは、証明者が検証者に対して、ある statement が真であることを納得させるための暗号プロトコルである。このとき証明者は、statement を真にしている秘密情報、すなわち

witness を持っているが、検証者にその witness を開示しない。検証者は、公開情報と証明のみを用いて statement の正しさを確認する。

典型的には、ZKP の statement は「公開インスタンス x に対して、ある秘密情報 w が存在し、関係 $R(x, w) = 1$ を満たす」という形で表現される。

$$\exists w \quad \text{s.t.} \quad R(x, w) = 1 \tag{1.1}$$

ここで、x は検証者にも公開される情報であり、w は証明者だけが知る秘密情報である。ZKP の役割は、そのような w を知っている、またはそのような w が存在することを、w 自体を漏らさずに示すことである。NIST も、技術的には、公開インスタンスと整合する秘密情報を知っていることを示す「proof of knowledge（知識の証明）」として ZKP を扱う場合が多いと説明している [2]。

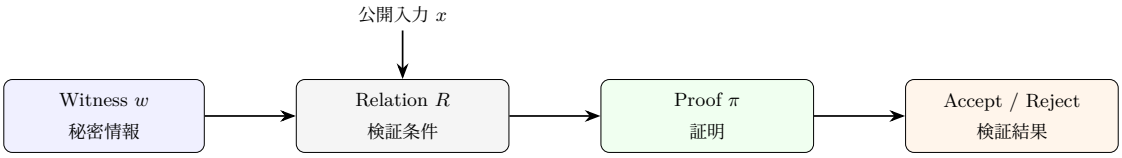


図1.1: ZKP の基本構造。witness は証明生成には使われるが、検証者へは開示されない。

ZKP が満たすべき基本性質は、一般に三つに整理される。第一に、完全性である。statement が真であり、証明者が適切な witness を持っている場合、正直な検証者は高い確率で証明を受理する。第二に、健全性である。statement が偽である場合、不正な証明者が検証者を欺いて受理させることは困難でなければならない。第三に、ゼロ知識性である。検証者は、statement が真であるという事実以外に、witness に関する実質的な追加情報を得ない。

実用上の多くの ZKP は、厳密な意味での proof ではなく argumentとして構成されている。暗号理論では、健全性が計算量的に制限された攻撃者に対してのみ成り立つ場合を argument と呼ぶ。zk-SNARK や Bulletproofs など、現代の多くの実用方式はこの枠組みに属する。したがって、方式評価では、「ゼロ知識」という語だけでなく、想定する攻撃者、計算量仮定、信頼セットアップ、証明サイズ、検証時間、証明生成時間を明確に確認する必要がある。

1.2 ゼロ知識証明の歴史

ZKP の歴史は、1980 年代の計算量理論における概念的発見から始まり、認証・署名への応用、非対話化、汎用計算証明、ブロックチェーン応用、透明セットアップ型方式、ポスト量子性を意識

した方式へと展開してきた。その発展過程は、単なる性能向上の歴史ではない。何を証明できるか、どのような信頼仮定に依存するか、検証者にどの程度の計算を要求するか、秘密情報をどの範囲で秘匿できるか、将来の量子計算機に対してどの程度の安全性を期待できるか、という複数の設計思想が分岐してきた歴史である。

ZKP の起点は、Shafi Goldwasser、Silvio Micali、Charles Rackoff による The Knowledge Complexity of Interactive Proof Systems である。同研究は、証明が検証者にどの程度の「知識」を与えるかを計算量理論の対象として定式化し、statement の正しさ以外の追加情報を与えない証明の概念を導入した [3]。この研究の意義は、証明という行為を、正しさの伝達だけでなく情報漏えいの制御という観点から捉え直した点にある。

続いて、Goldreich、Micali、Wigderson は、ZKP の一般性を大きく拡張した。同研究は、ZKP が特定の数論的問題に限られず、NP に属する広範な言語に対して構成可能であることを示した [4]。この段階で ZKP は、特定の暗号プロトコルではなく、広範な計算命題に適用可能な理論的枠組みとして位置づけられた。現在の汎用 ZKP、すなわち任意のプログラム、算術回路、制約システム、仮想マシン実行の正しさを証明する技術は、この理論的系譜の上にある。

1980 年代後半には、ZKP の考え方が認証・識別・署名へ接続された。Fiat-Shamir 方式は、識別と署名のための実用的方式を提示し、因数分解困難性に基づく効率的な構成を示した [5]。Schnorr 方式は、離散対数問題に基づく効率的な対話型識別方式と署名方式を提示し、スマートカードのような制約のあるデバイスに適した方式として位置づけられた [6]。この時期の研究は、証明者が秘密情報を知っていることを秘密情報を明かさずに示す ZKPoK の実用化につながった。

非対話型 ZKP も重要な展開である。Blum、Feldman、Micali は、共通ランダム文字列を共有することで対話を置き換えられることを示し、非対話型ゼロ知識証明の基礎を築いた [7]。その後、Fiat-Shamir 変換は、公開コイン型対話プロトコルのランダムチャレンジをハッシュ関数で置き換える技法として広く用いられるようになった。

2010 年代には、ZKP の実用化が大きく進んだ。Pinocchio は、クラウドに委託された一般計算の結果を検証するためのシステムとして提案され、検証可能計算を実用水準へ近づけた [8]。Zerocash は、zk-SNARK を用いて、支払元、支払先、送金額を秘匿しながら取引の正当性を検証する分散型匿名決済方式を構成した [9]。

2016 年には、Jens Groth による Groth16 が発表された。Groth16 は、ペアリングベースの非対話型 argument であり、証明が 3 つの群要素のみで構成され、検証が少数のペアリングで済む簡潔な方式である [10]。一方で、一般に回路ごとの信頼セットアップを必要とし、ペアリングベー

スの代数的仮定に依存するため、第 4 章では代数的仮定ベースの ZKP の代表例として評価する。

同じく 2010 年代には、用途特化型の ZKP・ZKPoK も発展した。Bulletproofs は、信頼セットアップを必要としない短い非対話型 ZKP として提案され、特にコミットされた値が一定範囲内にあることを示すレンジプルーフに適している [11]。One-out-of-Many proof は、多数のコミットメントのうち少なくとも 1 つについて特定の関係が成り立つことを、その対象がどれかを明かさずに証明する方式として提案された [12]。BBS 署名は、複数メッセージ署名と署名知識証明により、選択的開示と匿名資格証明の重要な基盤になっている [13, 14]。

信頼セットアップを不要とし、ポスト量子性を志向する透明セットアップ型方式も発展した。ZK-STARK は、transparent ZK system としての ZK-STARK を示し、post-quantum secure な computational integrity を掲げた [15]。Ligero は、信頼セットアップや公開鍵暗号を必要とせず、通信量が検証回路サイズの平方根に比例する軽量な ZK argument として提案された [16]。Aurora は R1CS 向けの transparent succinct argument として、Virgo は layered arithmetic circuit 向けの透明なゼロ知識 argument として提案された [17, 18]。

このように、ZKP の歴史は、知識複雑性の理論から始まり、汎用性、非対話性、簡潔性、実装性、匿名性、透明性、ポスト量子性へと展開してきた。本稿の方式比較は、この歴史的分岐を反映する。すなわち、Groth16、BBS 署名、Bulletproofs、One-out-of-Many proof、PLONK は、主として楕円曲線、ペアリング、離散対数、コミットメントなどに基づく代数的仮定ベースの方式群として整理できる。一方、ZK-STARK、Ligero、Aurora、Virgo は、透明セットアップ、ハッシュ関数、PCP、IOP、FRI、多項式検査、符号理論などを中核とする方式群として整理できる。

第 2 章　ゼロ知識証明の基本

ZKP は、ある statement が正しいことを、その statement を成り立たせる秘密情報を開示せずに検証可能にする暗号技術である。理解のためには、statement、witness、relation、完全性、健全性、ゼロ知識性、対話型と非対話型の違いを押さえる必要がある。本章では、それらの概念を整理し、第 3 章のセキュリティ定義と第 4 章の方式比較へつなげる。

2.1　ゼロ知識証明の意義

ZKP の意義は、大きく二つに整理できる。第一に、プライバシー保護と情報秘匿性である。第二に、検証・監査コストの削減と効率性である。前者は不要な情報を開示しないことに関わり、後者はすべてを再計算・再監査しなくても正しさを確認できることに関わる。

この二つは実務上密接に結びついている。たとえば、組織が一定の規制要件を満たしていることを示す場合、従来は内部統制資料、顧客データ、取引記録、監査証跡などを提示する必要があった。ZKP を用いれば、条件を満たすことだけを証明し、詳細な内部データの開示を抑制できる可能性がある。ISO/IEC 27565:2026 は、個人データの共有・送信に伴うリスクを低減し、不要な情報開示を最小化するための ZKP 利用ガイドラインとして公表されている [19]。

2.1.1　プライバシー保護と情報秘匿性

ZKP の直感的な価値は、情報開示の最小化である。年齢確認を例にすると、検証者が必要とするのは、利用者が閾値年齢以上であるという事実であって、生年月日、住所、本人確認書類番号などではない。ZKP を用いる場合、発行機関が署名した資格情報をもとに、「年齢が閾値以上である」という述語だけを証明し、生年月日そのものや他の属性を開示しない設計が可能になる。

この性質は、営業秘密や競争上の機密にも適用できる。価格条件、取引相手、在庫情報、信用スコア、AI モデルの重み、サプライチェーン上の調達先情報などは、開示すれば不利益を招き得る情報である。ZKP は、これらの情報を直接開示せずに、特定の条件や制約を満たしていることを証明する手段を提供する。

ただし、ZKP が隠すのは、プロトコル内で witness として扱われる情報である。公開入力、証

明提示のタイミング、検証者、ネットワーク情報、証明要求の頻度、複数証明の組合せなどから情報が推測される可能性は残る。したがって、ZKP の導入では、暗号プロトコルだけでなく、公開入力設計、属性設計、ログ設計、匿名性、リンク不能性、アクセス制御を含むシステム全体のプライバシー設計が必要である。

2.1.2 検証・監査コストの削減と効率性

ZKP の第二の意義は、検証・監査の効率化である。ここでいう効率性は、計算コストの一律削減ではない。多くの実用的 ZKP では、証明者側が証明生成のために相応の計算コストを負担する。特に汎用計算を証明する場合、証明生成は元の計算より高コストになることがある。

一方で、ZKP は、検証者側のコスト、通信量、保存量、監査コストを削減し得る。SNARK は、短い証明と高速な検証を特徴とする非対話型 argument of knowledge であり、検証者は秘密情報や計算過程を受け取らず、短い証明を検証することで statement の正しさを確認できる [20]。この構造は、クラウド計算、分散台帳、規制監査、公開証跡などに有用である。

たとえば、外部計算サービスが、指定されたプログラムを正しく実行し出力を得たことを証明する場合、依頼者は計算全体を再実行せずに証明を検証できる。Jolt のような zkVM 研究は、通常のプログラムを仮想マシン上で実行し、その実行トレースを SNARK で証明する方向を追求している [21]。

ZKP の効率性は、コストの再配分として理解すべきである。証明者は追加コストを負担するが、検証者、監査者、ネットワーク、データ管理者の負担は軽減され得る。方式選定では、証明者と検証者のどちらが計算負担を許容できるか、証明をどれだけ長期保存するか、検証がオンチェーンで行われるか、公開検証可能性が必要かを考慮する。

2.2 Statement と Witness

ZKP を理解するうえで最も重要な概念が、statement と witness である。statement は証明したい主張、witness はその主張が真であることを支える秘密情報である。暗号理論では、これを関係 R によって定式化する。

公開情報を x、秘密情報を w、両者の関係を R とすると、ZKP は次を証明する。

$$R(x, w) = 1. \tag{2.1}$$

このとき、x は検証者にも公開される入力であり、w は証明者が保持する秘密入力である。ZKP は、「ある w が存在し、$R(x, w) = 1$ を満たす」ことを、w 自体を明らかにせずに証明する。対応する言語を次のように定義する。

$$L_R = \{x \mid \exists w \text{ s.t. } R(x, w) = 1\}. \tag{2.2}$$

すなわち、公開入力 x が、有効な witness を持つ言語 L_R に属していることを証明する。

年齢確認では、statement は「発行機関が署名した資格情報に含まれる生年月日から計算した年齢が閾値以上である」という命題である。公開入力 x には、発行機関の公開鍵、現在日付、年齢条件、資格情報に対するコミットメントなどが含まれ得る。witness w には、生年月日、資格情報、発行機関の署名、コミットメントの乱数などが含まれ得る。

クラウド計算では、statement は「プログラム C を正しく実行し、出力 y を得た」という命題である。

$$\exists w \quad \text{s.t.} \quad C(x, w) = y. \tag{2.3}$$

この場合、公開入力はプログラム C、公開入力 x、出力 y であり、witness は秘密入力や計算途中の実行トレースである。

ZKP では、何を証明するかの設計が本質的である。statement の粒度が粗すぎると業務上必要な検証ができず、細かすぎると公開入力や証明結果から機密情報が推測される可能性がある。暗号方式の選定以前に、どの業務要件をどの statement として表現するかを検討する必要がある。

2.3 ゼロ知識証明の 3 つの特性

ZKP は、完全性、健全性、ゼロ知識性という三つの基本特性を満たす必要がある。ここでは直感的説明にとどめ、数学的定義は第 3 章で述べる。

完全性とは、statement が真であり、証明者が有効な witness を持ち、証明者と検証者がプロトコルに正直に従う場合、検証者が証明を受理する性質である。完全性は、正当な利用者や正当な事業者が不当に拒否されないための性質であり、実務上は可用性や業務継続性にも関わる。

健全性とは、statement が偽である場合、不正な証明者が検証者をだまして受理させることが困難であるという性質である。実用的な ZKP では、健全性が計算量的仮定のもとで成り立つ場合が多い。この場合、暗号理論上は proof ではなく argument と呼ばれる。

ゼロ知識性とは、検証者が、statement が真であるという事実以外に、witness に関する実質的

な情報を得ないという性質である。形式的には、実際の証明者とのやり取りで得られる情報と、witness を知らないシミュレータが生成する情報とが区別できないことによって定義される。

実務上は、これら三特性に加えて、知識健全性、シミュレーション抽出可能性、非可鍛性、同時実行安全性、リンク不能性、失効可能性などの追加性質が必要になる場合がある。特に、「statement が真である」だけでなく「証明者が実際に witness を知っている」ことを保証したい場合には、proof of knowledge としての性質が重要になる。

2.4 インタラクティブ証明と非インタラクティブ証明

ZKP には、証明者と検証者が複数回メッセージをやり取りするインタラクティブ証明と、証明者が単一の証明を生成し、検証者がそれを検証する非インタラクティブ証明がある。

インタラクティブ証明では、典型的に証明者がコミットメントを送り、検証者がランダムなチャレンジを返し、証明者がレスポンスを送る。このような三手順の構造を持つプロトコルは、Σ プロトコルと呼ばれることが多い。認証や二者間識別では、インタラクティブ証明が自然な場合がある。

非インタラクティブ証明では、証明者が一度だけ証明を生成し、検証者はその証明を受け取って受理または棄却する。この性質により、非インタラクティブ ZKP は、ブロックチェーン上の検証、監査ログ、資格証明、第三者検証、非同期ワークフローに適している。

対話型プロトコルを非対話化する代表的技法が Fiat-Shamir 変換である。公開コイン型プロトコルにおける検証者のランダムチャレンジを、ハッシュ関数から得られるチャレンジに置き換えることで、対話を除去する。ただし、ハッシュ関数のドメイン分離、公開入力の完全なバインディング、乱数生成、バージョン管理が不適切であると、実装に脆弱性が生じ得る。

非インタラクティブかつ公開検証可能な証明は複製・転送が容易である。これは監査やブロックチェーンでは利点であるが、認証や本人性確認では、リプレイ攻撃、証明の使い回し、リンク可能性に対する設計が必要になる。

2.5 現代的ゼロ知識証明の概観

現代的な ZKP は、複数の証明システム、暗号プリミティブ、算術化手法、コミットメント方式、署名知識証明、仮想マシン実行証明からなる技術群である。整理するうえで重要なのは、汎用計算証明と用途特化型証明の区別、安全性仮定、信頼セットアップ、ポスト量子性、証明対象の表現方

法である。

汎用計算証明は、任意の計算、プログラム、算術回路、制約システム、仮想マシン実行などを対象とする。Groth16、PLONK、ZK-STARK、Ligero、Aurora、Virgo は、この文脈で理解される。一方、用途特化型証明は、範囲証明、署名知識証明、集合所属証明、選択的開示、匿名性証明など、特定の statement に最適化されている。Bulletproofs、BBS 署名、One-out-of-Many proof はこの分類に入る。

安全性仮定の観点では、Groth16 や多くの PLONK 系 SNARK は、楕円曲線、ペアリング、構造化参照文字列、多項式コミットメントなど、代数的な暗号仮定に基づく。これに対して、ZK-STARK、Ligero、Aurora、Virgo は、ハッシュ関数、PCP、IOP、FRI、多項式検査、符号理論、軽量プリミティブなどを利用し、透明性とポスト量子性を重視する方向の方式群として理解できる。

信頼セットアップも重要である。信頼セットアップを必要とする方式では、セットアップ時に生成される秘密情報、いわゆる toxic wasteが適切に破棄されなければならない。これに対し、Bulletproofs、ZK-STARK、Ligero、Aurora、Virgo などは、信頼セットアップを不要とする、または透明セットアップを志向する方式として位置づけられる。

ポスト量子性は、長期的な暗号移行リスクの観点で重要である。NIST は 2024 年に FIPS 203、FIPS 204、FIPS 205 を公開し、ポスト量子暗号標準化を進めている [22]。ZKP においても、楕円曲線、離散対数、ペアリングに基づく方式は、現在の実用性に優れる一方、長期安全性の観点では注意が必要である。透明セットアップ・ハッシュ／ PCP・IOP ベースの方式は、ポスト量子性を期待し得る方式群として検討される。ただし、実際の安全性は、ハッシュ関数、パラメータ、Fiat-Shamir 変換、サウンドネスエラー、量子ランダムオラクルモデルでの解析に依存する。

表2.1: 現代的 ZKP の整理

分類軸	主な方式・技術	主な特徴
汎用計算証明	Groth16、PLONK、ZK-STARK、Ligero、Aurora、Virgo	任意の計算や制約充足を証明する。
用途特化型 ZKPoK	BBS 署名、Bulletproofs、One-out-of-Many proof	属性証明、範囲証明、集合所属、匿名性に強い。
代数的仮定ベース	Groth16、PLONK、BBS 署名、Bulletproofs、One-out-of-Many proof	短い証明や高速検証に強いが、量子耐性やセットアップが論点。
透明セットアップ・ハッシュ／ PCP・IOP ベース	ZK-STARK、Ligero、Aurora、Virgo	信頼セットアップ不要、公開鍵暗号不要、ポスト量子性を期待し得る方式群。
検証可能計算	SNARK、STARK、zkVM、recursive proof	クラウド計算、監査、ブロックチェーン、AI 推論に適用可能。

第 3 章　ゼロ知識証明のセキュリティ特性と評価軸

ZKP を評価するには、直感的な説明だけでは不十分である。ZKP は暗号プロトコルであり、その安全性は、完全性、健全性、ゼロ知識性という基本特性、および知識健全性、信頼セットアップ、計算量仮定、実装上の安全性によって定義される。本章では、まず数理的な安全性定義を整理し、そのうえでスキーム比較時の評価軸を提示する。

3.1　ゼロ知識証明におけるセキュリティ定義

3.1.1　形式化の前提：関係・言語・証明システム

ZKP で扱う問題を関係 R によって定式化する。関係 R は、公開入力 x と秘密入力 w の組に対して、真偽値を返す判定関係である。

$$R(x, w) \in \{0, 1\}. \tag{3.1}$$

$R(x, w) = 1$ であれば、w は x に対する有効な witness である。関係 R が定める言語 L_R は次で定義される。

$$L_R = \{x \mid \exists w \text{ s.t. } R(x, w) = 1\}. \tag{3.2}$$

非対話型証明システムは、概略的には次のアルゴリズムからなる。

$$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, R), \tag{3.3}$$

$$\pi \leftarrow \mathsf{Prove}(\mathsf{pp}, x, w), \tag{3.4}$$

$$b \leftarrow \mathsf{Verify}(\mathsf{pp}, x, \pi). \tag{3.5}$$

ここで、λ はセキュリティパラメータ、pp は公開パラメータ、π は証明、$b \in \{0, 1\}$ は検証結果である。対話型 ZKP の場合は、証明者 P と検証者 V が複数回メッセージを交換し、最終的に検証者が受理または棄却を出力する。

以下では、確率はセットアップ、証明者、検証者、攻撃者の乱数に関して取る。また、$\mathsf{negl}(\lambda)$ は無

視可能関数を表す。すなわち、任意の多項式 $p(\cdot)$ に対して、十分大きな λ では $\mathsf{negl}(\lambda) < 1/p(\lambda)$ となる関数である。

3.1.2 完全性

完全性とは、statement が真であり、証明者が有効な witness を持ち、証明者と検証者がプロトコルに正直に従う場合、検証者が証明を受理する性質である。非対話型証明システムでは、完全性は次のように表せる。

$$\Pr\left[\begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, R); \\ \pi \leftarrow \mathsf{Prove}(\mathsf{pp}, x, w); \\ \mathsf{Verify}(\mathsf{pp}, x, \pi) = 1 \end{array}\right] \geq 1 - \mathsf{negl}(\lambda), \tag{3.6}$$

ただし、すべての (x, w) について $R(x, w) = 1$ であるとする。完全性エラーが存在しない場合、すなわち上記の確率が常に 1 である場合を perfect completeness という。

完全性は、可用性や業務継続性と関係する。正当な証明者が高い確率で受理されなければ、暗号学的には安全でもシステムとして成立しない。証明生成時の乱数不足、端末性能不足、実装バグ、証明鍵のバージョン不一致などによって、理論上の完全性とは別に実装上の失敗が生じ得る。

3.1.3 健全性

健全性とは、statement が偽である場合、不正な証明者が検証者をだまして受理させることが困難であるという性質である。非対話型証明システムにおける標準的な健全性は、任意の不正な証明者または攻撃者 A に対して、次の確率が無視可能であることとして表せる。

$$\Pr\left[\begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, R); \\ (x, \pi) \leftarrow A(\mathsf{pp}); \\ x \notin L_R \ \wedge \ \mathsf{Verify}(\mathsf{pp}, x, \pi) = 1 \end{array}\right] \leq \mathsf{negl}(\lambda). \tag{3.7}$$

対話型証明では、不正な証明者 P^* が検証者 V と相互作用し、偽の statement $x \notin L_R$ を受理させる確率が無視可能であることを要求する。

$$\Pr[\langle P^*(x), V(x)\rangle = 1] \leq \mathsf{negl}(\lambda) \quad \text{for all } x \notin L_R. \tag{3.8}$$

健全性には、情報理論的な健全性と計算量的な健全性がある。任意の計算能力を持つ証明者に対しても偽の statement を受理させられない場合、その方式は proof と呼ばれる。一方、計算量

的に制限された証明者に対してのみ健全性が成り立つ場合、その方式は argument と呼ばれる。現代の多くの実用的 ZKP、特に SNARK 系の方式は、厳密には zero-knowledge proof ではなく zero-knowledge argument である [23]。

3.1.4 ゼロ知識性

ゼロ知識性とは、検証者が、statement が真であるという事実以外に、witness に関する実質的な情報を得ないという性質である。形式的には、実際の証明者との相互作用によって得られる検証者の view と、witness を知らないシミュレータが公開情報のみから生成する view とが識別不能であることによって定義される。

対話型 ZKP では、任意の確率的多項式時間の悪意ある検証者 V^* に対して、ある確率的多項式時間シミュレータ S が存在し、すべての $x \in L_R$ および対応する w について、次の二つの分布が識別不能であることを要求する。

$$\mathsf{View}_{V^*}^{P(x,w)}(1^\lambda, x) \approx_c S(1^\lambda, x). \tag{3.9}$$

ここで、$\approx_c$ は計算量的識別不能性を表す。すなわち、任意の確率的多項式時間識別器 D に対して、

$$\left|\Pr[D(\mathsf{View}_{V^*}^{P(x,w)}(1^\lambda, x)) = 1] - \Pr[D(S(1^\lambda, x)) = 1]\right| \leq \mathsf{negl}(\lambda) \tag{3.10}$$

が成り立つ。

この定義の直感は明確である。もし witness を知らないシミュレータが、実際の証明と区別できない view を生成できるなら、実際の証明プロトコルは witness に関する情報を漏らしていないとみなせる。

非対話型 ZKP では、実際の証明とシミュレートされた証明の分布が識別不能であることとして定義される。典型的には、シミュレーション用セットアップを用いて次のように表す。

$$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, R), \qquad \pi \leftarrow \mathsf{Prove}(\mathsf{pp}, x, w), \tag{3.11}$$

$$(\mathsf{pp}', \tau) \leftarrow \mathsf{SimSetup}(1^\lambda, R), \qquad \pi' \leftarrow \mathsf{SimProve}(\mathsf{pp}', \tau, x). \tag{3.12}$$

そして、すべての $x \in L_R$ について、

$$(\mathsf{pp}, \pi) \approx_c (\mathsf{pp}', \pi') \tag{3.13}$$

が成り立つことを求める。

ZKP が隠すのは witness に関する情報であり、公開入力 x に含めた情報は隠れない。公開入力設計が不適切であれば、プロトコルがゼロ知識であっても、公開入力から機密情報が推測される可能性がある。また、証明の提示時刻、検証者、通信経路、同一証明の再利用、証明対象の組合せなどのメタデータは、暗号プロトコルだけでは秘匿されない。

3.1.5 知識健全性、proof と argument、安全性の強度

実用的な ZKP では、完全性、健全性、ゼロ知識性に加えて、知識健全性が重要になる。知識健全性とは、証明者が検証者を納得させられるなら、その証明者は実際に有効な witness を「知っている」とみなせる性質である。形式的には、任意の不正な証明者 P^* に対して、ある抽出器 E^{P^*} が存在し、P^* が x について検証者を受理させるなら、E^{P^*} が $R(x, w) = 1$ を満たす witness w を抽出できることを要求する。

概略的には、受理される証明を生成できる攻撃者が存在するなら、その攻撃者から有効な witness を抽出できる、という考え方である。SNARK の最後の K、すなわち Knowledge は、この知識健全性に対応する。

安全性の強度には、完全安全性、統計的安全性、計算量的安全性がある。完全安全性は、エラーや識別可能性が数学的にゼロであることを意味する。統計的安全性は、無制限の計算能力を持つ攻撃者に対しても、分布間の統計的距離が無視可能であることを意味する。計算量的安全性は、確率的多項式時間の攻撃者に対してのみ安全性を要求する。方式評価では、何に対する安全性なのか、すなわち健全性エラー、ハッシュ衝突困難性、楕円曲線離散対数問題、ペアリングベース仮定、ランダムオラクルモデル、量子攻撃者の有無を明確化する必要がある。

3.2 スキーム比較時の評価軸

第 4 章では、代数的仮定ベースの方式として Groth16、BBS 署名、Bulletproofs、One-out-of-Many proof、PLONK を扱い、透明セットアップ・ハッシュ／ PCP・IOP ベースの方式として ZK-STARK、Ligero、Aurora、Virgo を扱う。これらは同じ ZKP という語で括られるが、証明対象、安全性仮定、証明サイズ、検証時間、セットアップ、ポスト量子性、実装成熟度が大きく異なる。

3.2.1 安全性仮定とポスト量子性

第一の評価軸は、安全性仮定である。代数的仮定ベースの方式では、楕円曲線離散対数問題、ペアリングベース仮定、Knowledge of Exponent 型仮定、多項式コミットメントの安全性などが重要になる。ハッシュ／ PCP・IOP ベースの方式では、衝突困難ハッシュ関数、ランダムオラクルモデル、FRI、多項式近接性検査、符号理論的検査などが中心になる。

ポスト量子性も重要である。楕円曲線、離散対数、ペアリングに基づく方式は、現在の実装効率に優れる一方で、将来の大規模量子計算機に対する長期安全性の観点では注意が必要である。ZK-STARK、Ligero、Aurora、Virgo のような透明セットアップ・ハッシュ／ PCP・IOP ベースの方式は、ポスト量子性を期待し得る方式群として扱われることが多い。ただし、ハッシュ関数ベースであるから自動的にポスト量子安全であるとは言えない。Fiat-Shamir 変換、ランダムオラクルモデル、量子ランダムオラクルモデル、サウンドネスエラー、ハッシュ出力長を含めて評価する必要がある。

3.2.2 信頼セットアップ

第二の評価軸は、信頼セットアップである。方式によっては、証明鍵や検証鍵を生成するために、共通参照文字列、構造化参照文字列、証明パラメータなどのセットアップを必要とする。信頼セットアップを必要とする方式では、生成時に生じる秘密情報が適切に破棄されなければならない。

表3.1: セットアップ分類

分類	説明	代表的論点
不要	特別な共通参照文字列を必要としない。	運用負荷が低いが、証明サイズや検証時間が大きくなる場合がある。
透明セットアップ	公開乱数やハッシュ関数など、秘密を含まないセットアップ。	toxic waste がないため監査しやすい。
信頼セットアップ	秘密を含む可能性のあるセットアップ。	秘密破棄、ceremony、監査責任が重要。
回路固有 SRS	各回路ごとにセットアップが必要。	Groth16 型で問題になりやすい。
ユニバーサル SRS	複数回路に使える SRS。	PLONK 系などで重要。
更新可能 SRS	後から参加者がランダム性を追加できる。	単一主体への信頼を緩和できる。

信頼セットアップは、暗号理論上の問題であると同時に、ガバナンス上の問題でもある。規制産業や公的監査が必要な用途では、セットアップ手続の透明性と証跡が導入判断に直結する。

3.2.3 証明サイズ・検証時間・証明生成時間

第三の評価軸は、性能である。ZKP の性能は、単一の指標では評価できない。証明サイズ、検証時間、証明生成時間、メモリ消費、セットアップ時間、検証鍵サイズ、証明鍵サイズ、通信量、ラウンド数を分けて評価する必要がある。

重要なのは、どの主体がどのコストを負担するかである。証明者が利用者端末である場合、証明生成時間とメモリ消費はユーザ体験に直結する。証明者がクラウド事業者である場合、証明生成コストはサービス提供コストに直結する。検証者がブロックチェーンやスマートコントラクトである場合、検証時間と証明サイズはオンチェーンコストに直結する。

3.2.4 証明対象の表現力と開発容易性

第四の評価軸は、証明対象の表現力である。ZKP は自然言語の命題を直接扱うのではなく、statement を数学的な制約に変換する必要がある。代表的な表現には、算術回路、R1CS、QAP、AIR、Plonkish 制約、lookup table、VM 実行トレースなどがある。

理論的性能よりも、証明対象を正しく実装できるかが重大な問題になる。年齢確認の statement で閾値年齢以上を証明する場合でも、発行者署名の検証、現在日付、失効確認、属性名のバインディング、タイムゾーン、境界条件を正しく回路化する必要がある。

3.2.5 プライバシー上の副次的性質

ゼロ知識性だけでなく、リンク不能性、witness indistinguishability、選択的開示、非転送性、指定検証者性、公開検証可能性も評価対象になる。

表3.2: プライバシー上の副次的性質

性質	説明	重要な用途
unlinkability	複数回の証明提示が同一主体に結びつかない。	匿名資格証明、年齢確認、ID 連携。
witness indistinguishability	複数の witness 候補のうち、どれを用いたか識別できない。	集合所属証明、匿名性証明。
selective disclosure	属性の一部だけを開示できる。	BBS 署名、Verifiable Credentials。
non-transferability	証明を第三者へ転送しても説得力を持ちにくい。	認証、本人性確認。
public verifiability	誰でも証明を検証できる。	監査、ブロックチェーン、公開証跡。

3.2.6 合成可能性・再利用耐性・実運用上の安全性

実際のシステムでは、ZKP は電子署名、コミットメント、認証基盤、資格証明、アクセス制御、監査ログ、ブロックチェーン、クラウド基盤と組み合わせられる。このとき、単独では安全な ZKP でも、複数プロトコルと組み合わせたときに安全性が維持されるとは限らない。リプレイ耐性、ドメイン分離、statement binding、context binding、乱数生成、サイドチャネル、鍵ローテーション、失効管理、監査証跡を評価する必要がある。

3.2.7 実装成熟度・標準化・監査可能性

ZKP 方式は、論文上の安全性と実装上の安全性に差が生じやすい。回路コンパイラ、フロントエンド言語、証明システム、楕円曲線ライブラリ、ハッシュ関数、乱数生成、GPU 実装、スマートコントラクト検証器など、多数の構成要素が関与するためである。評価では、参照実装、第三者監査、テストベクトル、相互運用性、パラメータ管理、依存関係、長期保守を確認する必要がある。

第 4 章　各 ZKP 方式の比較

ZKP は、異なる安全性仮定、証明対象、性能特性、運用要件を持つ証明システム群である。本章では、主要方式を二つの分類に分けて整理する。第一は、楕円曲線、ペアリング、離散対数、Pedersen コミットメントなどに基づく代数的仮定ベースの ZKP・ZKPoK である。第二は、信頼セットアップを不要とし、ハッシュ関数、PCP、IOP、FRI、多項式検査、符号理論などを利用する透明セットアップ・ハッシュ／ PCP・IOP ベースの ZKP である。

4.1　比較の前提：安全性仮定、用途、方式階層

4.1.1　汎用計算証明と用途特化型証明

方式比較では、汎用計算証明と用途特化型証明を区別する必要がある。汎用計算証明は、任意の計算、プログラム、算術回路、制約システム、仮想マシン実行などを対象とし、「この計算が正しく実行された」「この制約を満たす witness が存在する」といった statement を証明する。Groth16、PLONK、ZK-STARK、Ligero、Aurora、Virgo は、この文脈で理解できる。

用途特化型証明は、特定の statement に最適化された方式である。BBS 署名は、署名済み属性の一部を選択的に開示し、非開示属性を秘匿したまま署名の知識を証明する用途に適している。Bulletproofs は、コミットされた値が一定範囲内にあることを証明するレンジプルーフに強い。One-out-of-Many proof は、多数のコミットメントのうち 1 つについて秘密を知っていることを、その位置を明かさずに証明する用途に適している。

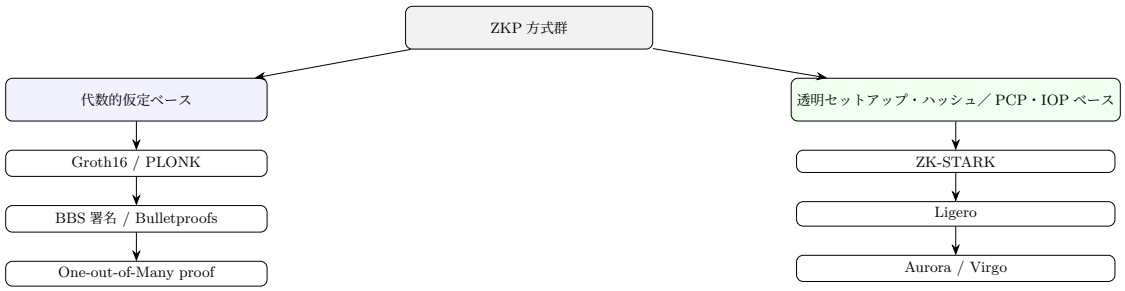


図4.1: 本稿で扱う ZKP 方式の分類。方式名は代表例であり、実際の派生方式はさらに多い。

4.1.2 ポスト量子性を評価する際の注意点

ポスト量子性は、第 4 章の重要な比較軸である。ただし、単純な二分法で扱うのは適切ではない。評価対象には、健全性、知識健全性、署名真正性、ゼロ知識性、ハッシュ関数仮定、Fiat-Shamir 変換、長期検証性が含まれる。たとえば BBS 署名の IETF CFRG ドラフトは、量子計算機が使用群の離散対数問題を破れる場合、署名者秘密鍵が導出され、署名真正性が成り立たなくなると説明している [13]。

本章では、楕円曲線・ペアリング・離散対数ベースの方式を原則として非ポスト量子方式として扱い、ZK-STARK、Ligero、Aurora、Virgo をポスト量子性を期待し得る透明セットアップ型方式として扱う。ただし後者も、具体的安全性は実装パラメータと安全性証明に依存する。

4.2 代数的仮定ベースの ZKP・ZKPoK

4.2.1 Groth16

Groth16は、ペアリングベースの preprocessing zk-SNARK であり、算術回路充足可能性に対して非常に短い証明と高速な検証を実現する代表的方式である。Groth16 の原論文は、証明が 3 つの群要素のみで構成され、検証は少数のペアリングを用いたペアリング積方程式の確認で済むことを示した [10]。

Groth16 の証明対象は、典型的には R1CS または QAP として表現される。R1CS では、変数ベクトルを $z = (1, x, w)$ とし、次の制約を満たすことを証明する。

$$(Az) \circ (Bz) = Cz, \tag{4.1}$$

ここで、$\circ$ は成分ごとの積であり、A, B, C は制約行列である。QAP 表現では、この制約充足性を多項式の割り切り条件として表す。

$$A_z(X)B_z(X) - C_z(X) = H(X)t(X). \tag{4.2}$$

Groth16 は、この多項式関係をペアリングによって簡潔に検証できる形へ変換する。

Groth16 の強みは、証明サイズと検証時間の小ささである。証明が数個の群要素で済むため、オンチェーン検証や多数の検証者への配布に有利である。一方、一般に回路ごとの信頼セットアップが必要であり、セットアップ時の toxic waste が漏えいすると不正な証明生成につながる可能性

がある。また、ペアリングベースの代数的仮定に依存するため、ポスト量子性は期待しにくい。

4.2.2 BBS 署名による署名知識証明

BBS 署名は、汎用 ZKP 方式ではなく、複数メッセージ署名と署名知識証明を組み合わせることで、選択的開示と匿名資格証明を実現する暗号プリミティブである。IETF CFRG の BBS 署名ドラフトは、複数メッセージに対する単一の定数サイズ署名を生成でき、署名保有者が署名済みメッセージの任意の部分集合を選択的に開示しつつ、署名のゼロ知識の知識証明を生成できると説明している [13]。W3C の Data Integrity BBS Cryptosuites も、BBS 署名を用いた選択的開示と unlinkable derived proofs を扱う [14]。

証明対象は概略的には次のように表せる。署名者の公開鍵を pk、署名を σ、開示するメッセージ集合を m_D、秘匿するメッセージ集合を m_H とすると、証明者は次を証明する。

$$\exists \sigma, m_H \quad \text{s.t.} \quad \mathsf{Verify}(pk, m_D \parallel m_H, \sigma) = 1. \tag{4.3}$$

検証者は、開示属性 m_D の真正性を確認できるが、秘匿属性 m_H や元の署名 σ は知る必要がない。

BBS 署名の強みは、Verifiable Credentials、年齢確認、資格証明、社内 ID、取引先資格確認と直結しやすいことである。一方で、任意計算を証明する方式ではなく、署名済み属性の選択的開示に特化している。また、現在の具体的 ciphersuite はペアリング曲線に依存するため、署名真正性についてポスト量子性は期待しにくい。乱数生成、失効管理、credential lifecycle、提示時の context binding も重要である。

4.2.3 Bulletproofs

Bulletproofsは、信頼セットアップを不要とする短い非対話型 ZKP であり、特にレンジプルーフに適している。原論文は、証明サイズが witness サイズに対して対数的であり、n ビットの範囲証明では $2\log_2(n) + 9$ 個の群・体要素で済むと示している [11]。

典型的な statement は、Pedersen コミットメントされた値 v が一定範囲内にあることを示すものである。

$$C = g^v h^\gamma, \qquad 0 \le v < 2^n. \tag{4.4}$$

これにより、取引金額、残高、担保率、年齢、スコアなどの数値属性について、条件を満たすこと

だけを示せる。

Bulletproofs の中核には内積証明がある。ベクトル $\mathbf{a}, \mathbf{b}$ に対して

$$\langle \mathbf{a}, \mathbf{b} \rangle = c \tag{4.5}$$

という関係を圧縮的に証明し、線形サイズの証明を対数サイズへ圧縮する。Bulletproofs は、信頼セットアップ不要性と範囲証明への適性が強みである。一方で、Groth16 のような極小証明や定数時間検証を提供するものではなく、離散対数仮定に基づくためポスト量子性は期待しにくい。

4.2.4 One-out-of-Many proof

One-out-of-Many proofは、多数のコミットメントのうち、少なくとも 1 つが特定の値に開くことを、その位置を明かさずに証明する方式である。Groth と Kohlweiss の方式は、少なくとも 1 つが 0 に開くコミットメント列に対して、そのどれであるかを明かさずに証明する 3 手の public-coin special honest verifier zero-knowledge な Σ プロトコルとして構成され、通信量がコミットメント数に対して対数的である [12]。

コミットメント列 $C_1, \ldots, C_N$ に対して、証明者はあるインデックス j と乱数 r_j を知っており、

$$C_j = \mathsf{Com}(0; r_j) \tag{4.6}$$

であることを証明する。ただし、検証者には j を明かさない。

この方式は、匿名性、集合所属、匿名アクセス制御、匿名コインなどに適用できる。汎用計算証明ではなく、集合所属・匿名性に特化した ZKPoK である。Pedersen コミットメントや離散対数設定に依存する場合、ポスト量子性は期待しにくい。

4.2.5 PLONK 系 SNARK

PLONKは、Groth16 と同じく代数的仮定ベースの SNARK であるが、回路ごとの信頼セットアップ負担を軽減する方向で重要な方式である。PLONK の原論文は、universal SNARK 構成と fully succinct verification を提示し、Lagrange 基底上の評価表現を用いることで permutation argument と arithmetization を簡潔に扱う [24]。

Plonkish 制約は、単純化すると各行 i について、ワイヤ値 a_i, b_i, c_i が次のゲート制約を満たす

ことを証明する。

$$q_L a_i + q_R b_i + q_O c_i + q_M a_i b_i + q_C = 0. \tag{4.7}$$

ここで、q_L, q_R, q_O, q_M, q_C は selector である。copy constraint は permutation argument で処理される。

PLONK 系の強みは設計上の柔軟性である。ユニバーサルまたは更新可能な SRS を利用できる構成が多く、複数の業務ロジックを同一基盤で証明したい場合に適しやすい。一方で、多くの場合、楕円曲線や多項式コミットメントに依存するため、ポスト量子性は期待しにくい。方式のバリエーションが多いため、具体的な実装、曲線、コミットメント方式、SRS 管理、監査状況を確認する必要がある。

4.3 透明セットアップ・ハッシュ／ PCP・IOP ベースの ZKP

透明セットアップ・ハッシュ／ PCP・IOP ベース方式は、信頼セットアップを不要とし、軽量暗号プリミティブや公開乱数に基づいて証明を構成する方式群である。一般に、ペアリングベース SNARK より証明サイズが大きくなりやすい一方、toxic waste を伴う信頼セットアップを避けられる。また、ハッシュ関数や符号理論、多項式検査を中心に構成されるため、ポスト量子性を期待し得る方式群として重要である。

4.3.1 ZK-STARK

ZK-STARKは、Scalable Transparent Argument of Knowledge の略であり、透明セットアップ、スケーラビリティ、ポスト量子性を重視する方式群である。Ben-Sasson らは、transparent ZK system としての ZK-STARK を報告し、検証がデータベースサイズに対して指数的に高速化することを掲げた [15]。

ZK-STARK では、計算を実行トレースとして表現し、そのトレースが低次数多項式に対応し、かつ遷移制約を満たすことを証明する。計算ステップ i における状態を T_i とすると、遷移制約は概略的に次のように書ける。

$$F_j(T_i, T_{i+1}) = 0. \tag{4.8}$$

これらを AIR として表現し、低次数性検査と FRIにより、検証者が全トレースを読むことなく正当性を確認できるようにする。

ZK-STARK の強みは、信頼セットアップ不要性とポスト量子性を期待し得る点である。一方で、一般に証明サイズが Groth16 などの SNARK より大きくなりやすい。AIR 設計、FRI パラメータ、ハッシュ関数選択、実行トレース生成など、実装上の複雑性も高い。

4.3.2 Ligero

Ligeroは、信頼セットアップを不要とする軽量な sublinear argument であり、透明セットアップ型 ZKP の重要な方式である。Ligero の拡張版論文は、NP に対するシンプルな ZK argument protocol を設計・実装し、通信量が検証回路サイズの平方根に比例することを述べている。また、任意の衝突困難ハッシュ関数に基づくことができ、ランダムオラクルモデルでは非対話型 zk-SNARK として利用でき、信頼セットアップや公開鍵暗号を必要としない [16]。

Ligero は、MPC-in-the-head 的な構成と interactive PCP を組み合わせた方式として理解できる。証明者が計算全体を符号化された形式でコミットし、検証者がランダムに一部を検査することで、全体の正しさを高い確率で保証する。

Ligero の強みは、構成が比較的単純で、信頼セットアップや公開鍵暗号を必要としないことである。一方で、Groth16 のような極小証明を提供するものではない。通信量は平方根オーダーであり、完全に succinct な SNARK と比べると大きくなる。非対話化する場合にはランダムオラクルモデルに依存する。

4.3.3 Aurora

Auroraは、R1CS 向けの透明な succinct argument である。原論文は、R1CS に対する zkSNARK を設計・実装・評価し、透明セットアップ、plausibly post-quantum secure、軽量暗号を特徴としている。n 個の制約の充足性を証明する証明サイズは $O(\log^2 n)$、証明生成は $O(n \log n)$ 個の体演算、検証は $O(n)$ とされる [17]。

Aurora の意義は、R1CS という既存 SNARK で広く使われる表現を、透明セットアップ型の証明へ接続した点にある。代数的 SNARK から透明型 ZKP への移行を考える際の橋渡しになる。一方で、検証時間が $O(n)$ とされる点には注意が必要であり、Groth16 のような極めて小さい検証コストを要求する用途では最適とは限らない。

4.3.4 Virgo

Virgoは、layered arithmetic circuit に対する透明な ZK argument であり、GKR 型プロトコルと透明な多項式委譲の系譜に位置づけられる。論文は、D 層、入力数 n、ゲート数 C の回路に対して、証明者時間 $O(C + n \log n)$、証明サイズ $O(D \log C + \log^2 n)$、構造化回路での検証時間 $O(D \log C + \log^2 n)$ を示している。また、衝突困難ハッシュ関数のような軽量プリミティブのみを用い、plausibly post-quantum secure であると説明している [18]。

Virgo の強みは、構造化された算術回路に対する効率性である。Merkle tree、ハッシュ木、反復的なデータ処理、一定構造を持つ計算に適しやすい。一方で、対象回路の構造に依存するため、すべての業務ロジックが効率的に表現できるわけではない。

4.4 各方式の横断比較

以下の表は、主要方式の性質を定性的に整理したものである。実際の性能は、回路サイズ、公開入力サイズ、ハッシュ関数、楕円曲線、セキュリティパラメータ、実装言語、ハードウェア、並列化、証明システムのバージョンによって大きく変わる。

表4.1: 主要 ZKP 方式の横断比較

方式	主分類	主な証明対象	信頼セットアップ	ポスト量子性	主な注意点
Groth16	ペアリングベース SNARK	汎用計算、R1CS/QAP	回路固有が一般的	期待しにくい	toxic waste、回路変更時の再セットアップ。
BBS 署名	署名知識証明	選択的開示、属性証明	ZKP 用 trusted setup は不要	署名真正性は期待しにくい	汎用計算証明ではない。失効管理が重要。
Bulletproofs	内積証明、レンジプルーフ	範囲証明、算術回路	不要	期待しにくい	検証時間・証明サイズは SNARK より重くなり得る。
One-out-of-Many	Σ プロトコル、集合所属	匿名集合所属	不要	期待しにくい	用途特化。非対話化設計が重要。
PLONK 系	ユニバーサル SNARK	汎用計算、Plonkish 制約	ユニバーサル／更新可能 SRS が多い	期待しにくい	実装差が大きい。SRS 管理が必要。
ZK-STARK	AIR/FRI 系	大規模計算、実行トレース	透明	期待し得る	証明サイズが大きくなりやすい。
Ligero	MPC-in-the-head/IPCP 系	NP、算術回路	不要	期待し得る	通信量は平方根オーダー。
Aurora	IOP/R1CS 系	R1CS	透明	期待し得る	検証時間や実装成熟度に注意。
Virgo	GKR/多項式委譲系	layered arithmetic circuit	透明	期待し得る	対象回路の構造依存性が高い。

安全性仮定の観点では、Groth16、PLONK、BBS 署名、Bulletproofs、One-out-of-Many proof

は、広義には代数的仮定ベースの方式群である。これらは現在の計算機環境では効率的に実装でき、証明サイズや検証時間に優れる場合が多い。一方で、楕円曲線、ペアリング、離散対数仮定に依存するため、長期的な量子耐性を重視する用途では慎重な評価が必要である。

これに対し、ZK-STARK、Ligero、Aurora、Virgo は、透明セットアップ、ハッシュ関数、PCP、IOP、多項式検査、符号理論を中心とする方式群である。既知の大規模量子計算機に対して相対的に耐性を期待し得る。ただし、Fiat-Shamir 変換を用いる非対話化やランダムオラクルモデルの扱い、ハッシュ出力長、soundness error、量子ランダムオラクルモデルでの解析を含めて評価する必要がある。

表4.2: 要件別の方式候補

要件	優先候補
極小証明・高速検証	Groth16
回路柔軟性・ユニバーサル SRS	PLONK 系
選択的開示・匿名資格証明	BBS 署名
範囲証明・金額秘匿	Bulletproofs
匿名集合所属	One-out-of-Many proof
透明セットアップ・大規模計算	ZK-STARK
公開鍵暗号不要・中規模回路	Ligero
R1CS と透明セットアップの両立	Aurora
構造化回路・多項式委譲	Virgo

方式選定は、最も安全な方式または最も高速な方式を一つ選ぶ作業ではない。証明したい statement、秘匿したい witness、検証主体、公開検証の必要性、信頼セットアップの許容性、長期安全性、実装成熟度、監査要件を総合して、用途ごとに適切な方式を選ぶ作業である。

第 5 章　ゼロ知識証明のユースケースと実用例

ZKP の応用は、秘密を隠す用途に限られない。実用上の価値は、秘匿された情報に基づく主張を、第三者が検証できる形に変換することにある。ユースケースを整理する際には、証明したい statement、秘匿したい witness、検証者が確認すべき事実、証明方式の四点から考えると見通しがよい。

5.1　デジタル ID・資格証明・選択的開示

デジタル ID は、ZKP の実用性が直感的に理解しやすい分野である。従来の本人確認や資格確認では、検証者が必要とする情報よりも多くの属性が提示されることが多い。年齢確認であれば、本来必要なのは閾値年齢以上であるという事実であり、生年月日、住所、本人確認書類番号、発行番号などのすべてを開示する必要はない。ZKP を用いると、発行者が署名した資格情報に基づき、特定の属性条件だけを証明し、他の属性を秘匿する設計が可能になる。

この領域では、W3C Verifiable Credentials Data Model が基盤となる。同仕様は、資格情報を暗号学的に安全で、プライバシーに配慮し、機械検証可能な形式で Web 上に表現するためのデータモデルを定義している [25]。W3C Data Integrity BBS Cryptosuites は、BBS 署名を用いた選択的開示と unlinkable proofs を扱う仕様である [14]。

典型的な statement は次のように表せる。

$$\exists \sigma, m_H \quad \text{s.t.} \quad \mathsf{Verify}(pk_I, m_D \parallel m_H, \sigma) = 1. \tag{5.1}$$

ここで、pk_I は発行者の公開鍵、σ は発行者署名、m_D は開示属性、m_H は秘匿属性である。検証者は、開示された属性が正当な発行者によって署名された資格情報の一部であることを確認できるが、秘匿属性や元の署名そのものを知る必要はない。

年齢確認では、次の statement を設計できる。

$$\exists\, \mathrm{credential}, \sigma, \mathrm{birthdate} \quad \text{s.t.} \quad \mathsf{Verify}(pk_I, \mathrm{credential}, \sigma) = 1 \wedge \mathsf{Age}(\mathrm{birthdate}, d_{\mathrm{today}}) \geq 20. \tag{5.2}$$

公開入力には発行者公開鍵、閾値年齢、検証日時、credential commitment などを含める。witness には、credential 本文、署名、生年月日、commitment の乱数を含める。

このユースケースでは、失効管理が特に重要である。資格情報が発行後に失効する場合、単に過去の署名を証明できるだけでは不十分である。失効リスト、status list、短期 credential、オンライン検証、匿名性を保った失効確認などを組み合わせる必要がある。

5.2 認証・アクセス制御

認証における ZKP の役割は、秘密情報そのものを提示せずに、秘密を知っていることを示すことである。古典的には、Schnorr 識別方式や Fiat-Shamir 型の識別プロトコルがこの系譜に属する。

簡略化した Schnorr 識別では、証明者は秘密値 x を持ち、公開値 $Y = g^x$ に対して、x を明かさずにその知識を示す。証明者は乱数 r を選び、

$$A = g^r \tag{5.3}$$

を送る。検証者がチャレンジ c を送り、証明者は

$$z = r + cx \tag{5.4}$$

を返す。検証者は

$$g^z = AY^c \tag{5.5}$$

を確認する。

パスワード認証の文脈では、SRP や J-PAKE のような PAKE プロトコルも重要である。RFC 2945 は SRP を、再利用パスワード問題を避けながら認証と鍵交換を行うプロトコルとして説明している [26]。RFC 8236 は J-PAKE を、両端が同じパスワードを知っているかどうか以外を学ばない形で鍵共有する PAKE として説明している [27]。

アクセス制御では、利用者が有効な権限を持つ、特定のグループに属する、ライセンス条件を満たす、有効期限内の資格情報を保持する、といった statement が対象になる。証明には、検証者

ID、セッション ID、nonce、時刻、要求対象リソースをバインドし、同じ証明が別セッションで再利用されないようにする必要がある。

5.3 金融・会計・監査

金融・会計領域では、ZKP の価値は、機密性の高い数値情報を開示せずに整合性や制約充足を示せる点にある。典型的な対象は、残高、取引金額、担保率、与信スコア、流動性比率、自己資本比率、特定上限を超えていないことなどである。

数値条件の証明では、レンジプルーフが基本となる。Pedersen コミットメント

$$C = g^v h^r \tag{5.6}$$

に対して、

$$0 \leq v < 2^n \tag{5.7}$$

を証明すれば、検証者は値 v を知らずに、その値が適切な範囲内にあることを確認できる。Bulletproofs は、このようなレンジプルーフに適している [11]。

暗号資産取引所などの準備金・支払能力証明も重要な応用である。Provisions は、取引所が Bitcoin アドレス、総保有額、総負債、顧客情報を開示せずに支払能力を証明する privacy-preserving proof of solvency として提案された [28]。Notus は、動的な proof of liabilities を扱い、一般的な負債更新に対して効率的な証明を提示する [29]。

金融・会計ユースケースでは、statement の定義が難しい。たとえば「支払能力がある」という statement は、資産、負債、評価時点、評価方法、対象口座、除外項目、為替レート、会計基準によって意味が変わる。ZKP は定義済みの statement を証明する技術であり、statement そのものの妥当性を保証するものではない。監査基準、会計ルール、データ範囲、時点情報、責任分界を事前に設計する必要がある。

5.4 ブロックチェーン・分散台帳

ブロックチェーン領域では、ZKP は二つの方向で利用されている。第一は、公開台帳上のプライバシー保護である。第二は、オフチェーン計算や状態遷移の正当性をオンチェーンで検証するスケーリング用途である。

プライバシー保護の代表例が Zerocash である。Zerocash は、zk-SNARK を用いて、支払元、支払先、送金額を秘匿しながら取引の正当性を検証する分散型匿名決済方式を構成した [9]。典型的な取引では、証明者は、コミットされた資産を正当に支出し、二重支払いを防ぐ nullifier を正しく生成し、台帳ルールを満たしていることを証明する。

スケーリング用途では、zk-rollup または validity rollup が代表例である。Ethereum の開発者向け資料は、zk-rollup において L2 がオフチェーン計算の validity proof を生成し、それをメインネットで検証する仕組みを説明している [30]。StarkEx の文書も、トランザクションをバッチ化し、SHARP に送ってバッチの妥当性を示す STARK proof を生成し、オンチェーン検証器で検証する流れを示している [31]。

状態遷移証明は、概略的には次の statement で表せる。

$$\exists T \quad \text{s.t.} \quad \mathsf{Execute}(S_{\text{old}}, \text{txs}) = S_{\text{new}} \wedge \mathsf{TraceValid}(T) = 1. \tag{5.8}$$

オンチェーン検証者は、各取引を再実行せずに、状態遷移の正しさを検証できる。

5.5 検証可能計算・クラウド委託計算

検証可能計算では、計算を実行した主体が、結果の正しさを証明する。利用者や依頼者は、計算全体を再実行せずに、短い証明を検証することで結果を受け入れられる。Pinocchio は、一般計算の検証可能性を実用水準へ近づけた代表的研究である [8]。

基本的な statement は次のように表せる。

$$\exists w \quad \text{s.t.} \quad C(x, w) = y. \tag{5.9}$$

ここで、C はプログラムまたは回路、x は公開入力、w は秘密入力または実行補助情報、y は出力である。検証者は、プログラム C が正しく実行され、出力 y が得られたことを確認する。

用途としては、外部計算サービスによる正当実行証明、データ分析結果の検証、契約で定められた計算手順の履行確認、データベース検索結果の完全性証明などがある。汎用計算を ZKP に載せるには、プログラムを算術回路、R1CS、AIR、Plonkish 制約、VM 実行トレースなどに変換する必要がある。比較、分岐、メモリアクセス、浮動小数点演算、暗号ハッシュ、署名検証、機械学習の非線形関数などは、証明生成コストに大きく影響する。

5.6 サプライチェーン・取引条件・契約遵守

サプライチェーンでは、取引先、調達先、検査結果、在庫量、製造条件、輸送条件、原産地、認証取得状況など、多くの機密情報が関係する。ZKP は、これらの情報をそのまま開示せずに、特定の条件を満たしていることを証明する用途に適用できる。

ある製品が特定の認証基準を満たす素材のみから作られていることを示す場合、公開入力には、認証機関の公開鍵、要求される基準、製品 ID、証明対象期間などを含める。witness には、調達記録、認証書、ロット情報、署名、コミットメント乱数などを含める。statement は次のように表せる。

$$\exists\, \mathrm{records}, \sigma_1, \ldots, \sigma_n \quad \text{s.t.} \quad \bigwedge_i \mathsf{Verify}(pk_i, \mathrm{record}_i, \sigma_i) = 1 \wedge \mathsf{Policy}(\mathrm{records}) = 1. \tag{5.10}$$

この領域では、ZKP 単体よりも、Verifiable Credentials、電子署名、台帳、監査ログ、IoT センサ署名、タイムスタンプと組み合わせる設計が現実的である。ZKP は、署名済み記録やコミットされたデータが条件を満たすことを証明できる。しかし、記録そのものが現実を正しく反映しているか、センサが改ざんされていないか、検査機関が信頼できるかは、別の仕組みで担保する必要がある。

5.7 電子投票・公開監査

電子投票は、ZKP の古典的かつ重要な応用分野である。投票システムでは、投票内容の秘匿と、集計結果の検証可能性を同時に満たす必要がある。ZKP は、暗号化された投票が有効な選択肢を表していること、同じ有権者が重複投票していないこと、暗号文のシャッフルや復号が正しく行われたことを示すために用いられる。

Helios は、Web ベースの open-audit voting system として提案された代表的なシステムであり、暗号学的監査によって投票プロセス全体を検証可能にする方向を示した [32]。ElectionGuard は、従来型の選挙システムと組み合わせて end-to-end verifiability を実現するためのオープンソースツール群である [33]。

候補者数が k で、投票ベクトルを $\mathbf{v} = (v_1, \ldots, v_k)$ とする。1 人 1 票制では、暗号化された投

票が次を満たすことを証明する必要がある。

$$v_i \in \{0,1\} \quad \text{for all } i, \tag{5.11}$$

$$\sum_{i=1}^{k} v_i = 1. \tag{5.12}$$

検証者は、投票内容を知らずに、投票が有効な形式であることを確認できる。

電子投票は、プライバシー、個別検証可能性、普遍検証可能性、強制耐性、投票買収耐性、可用性、ユーザビリティを同時に考慮する必要がある。ZKP は中心的な構成要素であるが、投票システム全体の安全性は、端末、認証、投票者登録、監査手順、運用体制にも依存する。

5.8 AI・機械学習：zkML

AI・機械学習では、ZKP を用いて、モデルや入力を秘匿しながら推論または学習の正しさを証明する研究が進んでいる。この領域は一般に zkMLと呼ばれる。2025 年のサーベイは、ZKML 研究を、verifiable training、verifiable inference、verifiable testing の三分類で整理している [34]。

推論の典型的な statement は次のように表せる。

$$\exists x, \theta \quad \text{s.t.} \quad f_\theta(x) = y \wedge \mathsf{Commit}(x) = C_x \wedge \mathsf{Commit}(\theta) = C_\theta. \tag{5.13}$$

ここで、x は入力、θ はモデルパラメータ、y は出力である。入力を秘匿したい場合は x を witness にし、モデルを秘匿したい場合は θ を witness にする。

ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs は、現実的な ML モデルに対して ZK-SNARK を生成するシステムを提示し、視覚モデルや distilled GPT-2 などを対象にしている [35]。実用化では、非線形関数、浮動小数点演算、行列演算、正規化、量子化、softmax、活性化関数などが課題になる。ZKP は有限体上の制約として計算を表現するため、通常の ML 計算をそのまま証明対象にすることは難しい。モデルの量子化、近似関数、lookup argument、専用回路、zkVM、operator-level proof などを組み合わせる必要がある。

5.9 ユースケース別の方式選定

各ユースケースに適した方式は、証明対象によって大きく異なる。以下は、主要ユースケースと候補方式の対応である。

表5.1: ユースケース別の方式選定

ユースケース	主な statement	秘匿される witness	候補方式
年齢確認	閾値年齢以上である。	生年月日、credential、署名。	BBS 署名、SNARK。
資格証明	有効な資格を保持する。	資格属性、署名、識別子。	BBS 署名、匿名資格証明。
アクセス制御	権限条件を満たす。	所属、権限、社員 ID。	BBS 署名、Schnorr 系 ZKPoK。
パスワード認証	同じ秘密を知っている。	パスワード。	PAKE、SRP、J-PAKE。
金額秘匿	金額が範囲内である。	金額、乱数。	Bulletproofs、SNARK。
支払能力証明	資産が負債以上である。	資産内訳、負債内訳、顧客情報。	SNARK、Pedersen commitment、Merkle tree。
匿名決済	取引が正当で二重支払いでない。	note、秘密鍵、金額、受取先。	zk-SNARK、STARK。
zk-rollup	状態遷移が正しい。	実行トレース、一部入力。	Groth16、PLONK、STARK。
クラウド計算検証	指定計算が正しく実行された。	秘密入力、実行補助情報。	SNARK、STARK、Aurora、Virgo。
サプライチェーン証明	policy を満たす記録が存在する。	取引先、数量、価格、検査記録。	VC、BBS 署名、SNARK。
電子投票	投票が有効で集計が正しい。	投票内容、シャッフル置換。	Σ プロトコル、NIZK、mix-net proof。
zkML	推論・学習が正しく行われた。	入力、モデル、重み、特徴量。	SNARK、STARK、lookup 系、zkVM。

この表から分かるように、ZKP の方式選定は、最も高性能な方式を選ぶ作業ではない。属性証明では BBS 署名が自然であり、数値範囲では Bulletproofs が適する場面が多い。オンチェーン検証では Groth16 や PLONK が有利な場合がある。透明セットアップや長期安全性を重視する場合は STARK 系や Ligero、Aurora、Virgo が候補になる。汎用計算や AI 推論では、証明対象の回路化または VM 化の容易さが大きな決定要因となる。

5.10 実装パターンと設計上の論点

ZKP のユースケースは多様であるが、実装パターンは、資格証明、範囲証明、計算正当性証明、集合所属・匿名性、集約・ロールアップに整理できる。

資格証明パターンでは、発行者が credential に署名し、保有者が必要な属性のみを開示または証明する。範囲証明パターンでは、値をコミットし、その値が所定範囲内にあることを証明する。計算正当性証明パターンでは、プログラムまたは回路 C が入力 x, w に対して出力 y を返すことを証明する。集合所属パターンでは、証明者が集合内のある要素に対応する秘密を知っていることを示すが、その要素がどれかは明かさない。集約・ロールアップパターンでは、多数の取引やイベントをまとめ、集約後の状態遷移が正しいことを証明する。

実用化に向けては、暗号方式の選択だけでなく、公開入力、freshness、失効管理、実装監査、説明可能性、監査可能性が重要である。公開入力に含めた情報は隠れないため、公開入力の粒度が不適切であれば、証明そのものがゼロ知識であっても機密情報が推測される。認証やアクセス制御では、nonce、challenge、session ID、expiration を証明に含める必要がある。資格証明では、発行後の権限喪失や資格失効を扱う仕組みが必要である。

ZKP の安全性は、証明システムだけでなく、回路実装、乱数生成、ハッシュ入力、ドメイン分離、鍵管理、SRS 管理、失効リスト、検証器実装にも依存する。IETF の BBS 署名ドラフトは、proof 生成時の乱数が再利用・予測可能・不適切生成である場合、非開示メッセージや署名値が危殆化し得ると警告している [13]。

以上を踏まえると、ZKP の実用化は、暗号プロトコルの導入というより、検証可能性と情報最小化を中心に据えた業務設計である。ユースケースごとに、statement、witness、公開入力、証明方式、検証者、失効管理、ログ設計、運用責任を対応づけることが有効である。

参考文献

[1] National Institute of Standards and Technology. *Zero-Knowledge Proof - CSRC Glossary*. 2026. URL: https://csrc.nist.gov/glossary/term/zero_knowledge_proof (visited on 04/27/2026).

[2] National Institute of Standards and Technology. *Zero-Knowledge Proof (ZKP) - Privacy-Enhancing Cryptography*. 2026. URL: https://csrc.nist.gov/projects/pec/zkproof (visited on 04/27/2026).

[3] S. Goldwasser, S. Micali, and C. Rackoff. "The Knowledge Complexity of Interactive Proof Systems". In: *SIAM Journal on Computing* 18.1 (1989), pp. 186–208. DOI: 10.1137/0218012.

[4] O. Goldreich, S. Micali, and A. Wigderson. "Proofs that Yield Nothing but Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems". In: *Journal of the ACM* 38.3 (1991), pp. 690–728. DOI: 10.1145/116825.116852.

[5] A. Fiat and A. Shamir. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *Advances in Cryptology - CRYPTO '86*. 1986, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.

[6] C.-P. Schnorr. "Efficient Identification and Signatures for Smart Cards". In: *Advances in Cryptology - CRYPTO '89*. 1989, pp. 239–252. DOI: 10.1007/0-387-34805-0_22.

[7] M. Blum, P. Feldman, and S. Micali. "Non-interactive Zero-Knowledge and Its Applications". In: *Proceedings of the 20th Annual ACM Symposium on Theory of Computing*. 1988, pp. 103–112. DOI: 10.1145/62212.62222.

[8] B. Parno, J. Howell, C. Gentry, and M. Raykova. *Pinocchio: Nearly Practical Verifiable Computation*. Cryptology ePrint Archive, Paper 2013/279. 2013. URL: https://eprint.iacr.org/2013/279.

[9] E. Ben-Sasson, A. Chiesa, M. Green, E. Tromer, and M. Virza. "Zerocash: Decentralized Anonymous Payments from Bitcoin". In: *2014 IEEE Symposium on Security and Privacy*. 2014, pp. 459–474. DOI: 10.1109/SP.2014.36.

[10] J. Groth. *On the Size of Pairing-Based Non-interactive Arguments*. Cryptology ePrint Archive, Paper 2016/260. 2016. URL: https://eprint.iacr.org/2016/260.

[11] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. *Bulletproofs: Short Proofs for Confidential Transactions and More*. Cryptology ePrint Archive, Paper 2017/1066. 2017. URL: https://eprint.iacr.org/2017/1066.

[12] J. Groth and M. Kohlweiss. *One-out-of-Many Proofs: Or How to Leak a Secret and Spend a Coin*. Cryptology ePrint Archive, Paper 2014/764. 2014. URL: https://eprint.iacr.org/2014/764.

[13] T. Looker, V. Kalos, A. Whitehead, and M. Lodder. *The BBS Signature Scheme, draft-irtf-cfrg-bbs-signatures-10*. 2026. URL: https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/ (visited on 04/27/2026).

[14] World Wide Web Consortium. *Data Integrity BBS Cryptosuites v1.0*. 2026. URL: https://www.w3.org/TR/vc-di-bbs/ (visited on 04/27/2026).

[15] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Paper 2018/046. 2018. URL: https://eprint.iacr.org/2018/046.

[16] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam. "Ligero: Lightweight Sublinear Arguments Without a Trusted Setup". In: *Designs, Codes and Cryptography* (2023). DOI: 10.1007/s10623-023-01222-8. URL: https://link.springer.com/article/10.1007/s10623-023-01222-8.

[17] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward. *Aurora: Transparent Succinct Arguments for R1CS*. Cryptology ePrint Archive, Paper 2018/828. 2018. URL: https://eprint.iacr.org/2018/828.

[18] J. Zhang and T. Xie. *Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof*. Cryptology ePrint Archive, Paper 2019/1482. 2019. URL: https://eprint.iacr.org/2019/1482.

[19] ISO/IEC. *ISO/IEC 27565:2026: Guidelines on privacy preservation based on zero-knowledge proofs*. 2026. URL: https://www.iso.org/standard/80398.html (visited on 04/27/2026).

[20] ZKProof Community. *ZKProof Community Reference*. 2022. URL: https://docs.zkproof.org/reference.pdf (visited on 04/27/2026).

[21] A. Arun, S. Setty, and J. Thaler. *Jolt: SNARKs for Virtual Machines via Lookups*. Cryptology ePrint Archive, Paper 2023/1217. 2023. URL: https://eprint.iacr.org/2023/1217.

[22] National Institute of Standards and Technology. *Post-Quantum Cryptography Standardization*. 2025. URL: https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization (visited on 04/27/2026).

[23] J. Thaler. *Proofs, Arguments, and Zero-Knowledge*. 2023. URL: https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.pdf (visited on 04/27/2026).

[24] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge*. Cryptology ePrint Archive, Paper 2019/953. 2019. URL: https://eprint.iacr.org/2019/953.

[25] World Wide Web Consortium. *Verifiable Credentials Data Model v2.0*. 2025. URL: https://www.w3.org/TR/vc-data-model-2.0/ (visited on 04/27/2026).

[26] T. D. Wu. *RFC 2945: The SRP Authentication and Key Exchange System*. 2000. URL: https://datatracker.ietf.org/doc/html/rfc2945 (visited on 04/27/2026).

[27] F. Hao. *RFC 8236: J-PAKE: Password-Authenticated Key Exchange by Juggling*. 2017. URL: https://www.rfc-editor.org/rfc/rfc8236.html (visited on 04/27/2026).

[28] G. G. Dagher, B. Bünz, J. Bonneau, J. Clark, and D. Boneh. "Provisions: Privacy-Preserving Proofs of Solvency for Bitcoin Exchanges". In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 2015, pp. 720–731. DOI: 10.1145/2810103.2813674.

[29] J. Xin, A. Haghighi, X. Tian, and D. Papadopoulos. "Notus: Dynamic Proofs of Liabilities from Zero-knowledge RSA Accumulators". In: *33rd USENIX Security Symposium*. 2024, pp. 5485–5502. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/xin.

[30] Ethereum Foundation. *Zero-Knowledge Rollups*. 2026. URL: https://ethereum.org/developers/docs/scaling/zk-rollups/ (visited on 04/27/2026).

[31] StarkWare. *StarkEx Documentation: High-level Overview*. 2026. URL: https://docs.starkware.co/starkex/overview.html (visited on 04/27/2026).

[32] B. Adida. "Helios: Web-Based Open-Audit Voting". In: *17th USENIX Security Symposium*. 2008, pp. 335–348. URL: https://www.usenix.org/legacy/event/sec08/tech/full_papers/adida/adida.pdf.

[33] J. Benaloh, M. Naehrig, O. Pereira, and D. S. Wallach. "ElectionGuard: A Cryptographic Toolkit to Enable Verifiable Elections". In: *33rd USENIX Security Symposium*. 2024, pp. 5485–5502. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/benaloh.

[34] Z. Peng, T. Wang, C. Zhao, G. Liao, Z. Lin, Y. Liu, B. Cao, L. Shi, Q. Yang, and S. Zhang. *A Survey of Zero-Knowledge Proof Based Verifiable Machine Learning*. arXiv:2502.18535. 2025. URL: https://arxiv.org/abs/2502.18535.

[35] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang. "ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs". In: *Proceedings of the Nineteenth European Conference on Computer Systems*. 2024, pp. 560–574. DOI: 10.1145/3627703.3650088.

索引

AIR, 16
Argument, 2
Aurora, 23

BBS 署名, 20
Bulletproofs, 20

完全性, 2

FRI, 22

Groth16, 19

インタラクティブ証明, 8

知識健全性, 14

Ligero, 23

非インタラクティブ証明, 8

One-out-of-Many proof, 21

PLONK, 21

Proof of Knowledge, 2
Prover, 1

QAP, 16

R1CS, 16

Σ プロトコル, 8
健全性, 2
Statement, i

Toxic waste, 9
信頼セットアップ, 15

Verifier, 1
Virgo, 24

Witness, i

ゼロ知識性, 2
ゼロ知識証明, i
ZK-STARK, 22
zkML, 31